

QUALIFICAÇÃO PROFISSIONAL — PROGRAMADOR FRONT-END

Desenvolvimento Web

Guia de Estudos — Formulários e Requisições a uma API

Continuação do projeto `Routes`. Além dos conceitos já vistos em aula, este guia explica com calma o que é uma API, o que significa "código assíncrono", e o que `fetch`, `async` e `await` realmente fazem — sem entrar em detalhes de back-end, só o suficiente para você entender o que está acontecendo por trás do código que escrevemos juntos.

CSS Modules • Formulários • API • Assíncrono • `useEffect`

Sumário

1. **CSS Modules** — evitando conflito de classes entre componentes
2. **Estilo Inline com Objetos**
3. **Formulários Controlados** — `value` + `onChange`
4. **Validação em Tempo Real**
5. **O que é uma API** — cliente, servidor e requisição
6. **JSON** — o formato dos dados que a API devolve
7. **Código Síncrono x Assíncrono** — por que precisamos esperar
8. `async` e `await` — escrevendo espera de forma legível
9. `useEffect` — rodando código fora da renderização normal
10. **Buscando Dados de uma API Real** — dados, carregando, erro
11. **API + Rotas** — buscando por parâmetro

1. CSS Modules

Evitando que o CSS de um componente afete outro sem querer.

Até agora, todo CSS que escrevemos é **global** — mesmo importando `./styles.css` dentro de um componente específico, as classes ali dentro ficam disponíveis para a aplicação **inteira**. Se dois componentes diferentes usarem a classe `.card` com estilos diferentes, um vai **sobrescrever** o outro sem querer.

Analogia

Pense em duas pessoas numa empresa com o mesmo apelido, "Duda". Toda vez que alguém grita "Duda!", as duas se viram — ninguém sabe qual delas foi chamada.

CSS Modules é uma convenção (já suportada pelo Vite) que torna as classes de um arquivo CSS **locais** àquele componente.

Passo a passo

1. Nomeie o arquivo terminando em `.module.css`:

```
src/pages/Produtos.module.css
```

2. Escreva o CSS normalmente:

```
.card {  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  padding: 12px;  
}
```

3. Importe como um objeto:

```
import styles from "./Produtos.module.css";
```

4. Use `styles.nomeDaClasse` no `className`:

```
<div className={styles.card}>...</div>
```

Prova de que funciona: inspecionando o HTML no navegador, a classe aparece como algo tipo `_card_1egpo_1` — um nome único gerado automaticamente, evitando colisão com qualquer outro componente.

Erro comum

Importar como `import "./Produtos.module.css"` (igual ao CSS normal) em vez de `import styles from "./Produtos.module.css"`.

2. Estilo Inline com Objetos

Quando faz sentido usar em vez de CSS Modules.

```
<div style={{ backgroundColor: corDeFundo, padding: "12px" }}>
```

	CSS Modules / <code>className</code>	Estilo inline
Quando usar	Estilo fixo , que não muda	Estilo que depende de uma variável
Nomenclatura	Normal (<code>background-color</code>)	camelCase (<code>backgroundColor</code>)

Regra prática

Se o estilo é sempre igual, ele pertence ao CSS. Se depende de um valor que muda em tempo de execução, o estilo inline se justifica.

3. Formulários Controlados

O valor do campo mora no estado do React, não no HTML.

Um **input controlado** é aquele cujo valor é gerenciado pelo **estado do React**.

```
import { useState } from "react";

function Formulario() {
  const [nome, setNome] = useState("");

  return (
    <input
      type="text"
      value={nome}
      onChange={(e) => setNome(e.target.value)}
    />
  );
}
```

Analogia

Um input "solto" é como uma caixa de sugestões: você escreve e ninguém sabe o que tem lá até abrir depois. Um input controlado é como ditar para alguém que escreve exatamente o que você fala — o estado sabe o valor o tempo todo.

Onde isso entra no projeto

src/pages/Contato.jsx (editar o arquivo inteiro)

```

import { useState } from "react";

export default function Contato() {
  const [nome, setNome] = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    console.log("Enviado:", nome, email);
  }

  return (
    <div>
      <h1>Fale conosco</h1>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Seu nome"
          value={nome}
          onChange={(e) => setNome(e.target.value)}
        />
        <input
          type="email"
          placeholder="Seu e-mail"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <button type="submit">Enviar</button>
      </form>
    </div>
  );
}

```

Nada muda no `App.jsx` — a rota `/contato` já existe desde a Aula 1.

4. Validação em Tempo Real

Mostrando erros conforme o usuário digita.

src/pages/Contato.jsx (mesmo arquivo, adicionando validação)

```
const emailValido = email.includes("@");

...

{email.length > 0 && !emailValido && (
  <p style={{ color: "red" }}>Digite um e-mail válido.</p>
)}
```

Erro comum

Esquecer a checagem de campo vazio e mostrar a mensagem de erro **antes** do usuário digitar qualquer coisa.

5. O que é uma API

Antes de escrever código, entender o que está acontecendo.

Até agora, todos os dados que usamos nos projetos foram escritos **à mão**, direto no código (arrays de produtos, por exemplo). Uma aplicação real busca dados de um lugar externo — um servidor, em algum lugar da internet, que guarda e devolve informações. A "porta de entrada" para conversar com esse servidor é a **API** (Application Programming Interface).

Analogia

Pense num restaurante. Você (o front-end) não entra na cozinha para pegar a comida — você faz um **pedido** ao garçom (a API), que leva até a cozinha (o servidor), espera o prato ficar pronto, e traz de volta a **resposta** (os dados) até a sua mesa. Você nunca precisa saber como a cozinha funciona por dentro — só precisa saber pedir e esperar a resposta chegar.

Requisição e resposta

Cada "pedido" ao servidor é chamado de **requisição** (request), e o servidor devolve uma **resposta** (response). No navegador, fazemos isso com a função `fetch`:

```
fetch("https://jsonplaceholder.typicode.com/users")
```

O texto entre aspas é o **endereço** da informação que queremos — parecido com uma URL comum que você digita no navegador, só que, em vez de devolver uma página, devolve dados.

Existem diferentes "tipos" de pedido, chamados **métodos HTTP**. Por enquanto, vamos usar só o `GET` — que significa "quero **buscar** uma informação, sem alterar nada no servidor" (nas próximas aulas veremos `POST`, `PUT` e `DELETE`, que enviam, atualizam e apagam dados).

Fica tranquilo com o "por trás dos panos"

Este curso é de front-end — você não precisa entender como o servidor guarda ou processa os dados, só precisa saber **pedir** (com `fetch`) e **usar a resposta** quando ela chegar.

6. JSON

O formato dos dados que a API devolve.

JSON (JavaScript Object Notation) é um formato de texto para representar dados — e a boa notícia é que ele se parece muito com os objetos e arrays que você já usa em JavaScript:

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "leanne@example.com"
}
```

A API quase sempre devolve os dados como texto em formato JSON — por isso, depois de receber a resposta, precisamos **converter** esse texto para um objeto/array JavaScript de verdade. É isso que o método `.json()` faz (você vai ver ele sendo usado já na próxima seção).

Por que isso importa

Repare que a estrutura é a mesma que já usamos nos arrays de produtos e filmes (`{ id: "1", nome: "..."}`) — só que, em vez de você escrever esses dados à mão no código, eles vêm prontos do servidor.

7. Código Síncrono × Assíncrono

Por que precisamos "esperar" a resposta.

Até hoje, todo código que escrevemos roda **síncrono**: uma linha só começa depois que a anterior terminou, e isso acontece quase instantaneamente.

Uma requisição para um servidor é diferente: ela pode demorar — décimos de segundo, às vezes mais, dependendo da internet e do servidor. Se o JavaScript **parasse tudo** esperando a resposta chegar, a tela travaria: nenhum clique, nenhuma animação, nada funcionaria enquanto isso.

A solução é código **assíncrono**: o JavaScript dispara a requisição, **não espera parado**, continua executando outras coisas, e só volta a rodar o código relacionado à resposta **quando ela chegar**.

Analogia

É como colocar uma pizza no forno (disparar a requisição) e, enquanto ela assa, você continua arrumando a mesa, atendendo o telefone, etc. (o resto da aplicação continua funcionando). Quando o timer toca (a resposta chega), você volta a dar atenção à pizza.

`fetch(...)` já é uma função assíncrona por natureza — ela devolve algo chamado **Promise** ("promessa"): um valor que ainda não existe, mas que *promete* existir em algum momento no futuro, quando a requisição terminar.

Não precisa decorar os detalhes da Promise

Para este curso, basta saber que `fetch` devolve uma "promessa de resposta futura", e que `async / await` (próxima seção) é a ferramenta que usamos para **esperar** essa promessa se cumprir, de um jeito fácil de ler.

8. `async` e `await`

Escrevendo código assíncrono como se fosse síncrono.

`await` significa "espere aqui até esse valor chegar, antes de continuar para a próxima linha" — mas sem travar o restante da aplicação, como explicamos na seção anterior.

```
async function buscarUsuarios() {  
  const resposta = await fetch("https://jsonplaceholder.typicode.com/users");  
  const dados = await resposta.json();  
  console.log(dados);  
}
```

Explicando bloco a bloco

- `async function buscarUsuarios()` — a palavra `async` antes de `function` é obrigatória para poder usar `await` dentro dela. Ela avisa o JavaScript: "esta função vai esperar coisas assíncronas por dentro".
- `await fetch(...)` — dispara a requisição e pausa apenas esta função até a resposta chegar (o resto da aplicação continua funcionando normalmente enquanto isso).
- `await resposta.json()` — converter de JSON para objeto JavaScript também demora um pouquinho e também é assíncrono, por isso tem seu próprio `await`.

Regra importante

`await` só pode ser usado **dentro** de uma função marcada com `async`. Usar `await` fora disso dá erro de sintaxe.

Por que isso é melhor que escrever "na mão" a espera? Sem `async / await`, o mesmo código ficaria assim (usando `.then()`, uma forma mais antiga e mais difícil de ler):

```
fetch("https://jsonplaceholder.typicode.com/users")  
  .then((resposta) => resposta.json())  
  .then((dados) => console.log(dados));
```

`async / await` faz o mesmo trabalho, mas lendo de cima para baixo, como qualquer outro código — por isso é a forma preferida hoje em dia.

9. `useEffect`

Rodando código fora do fluxo normal de renderização.

Sabendo agora o que é uma requisição assíncrona, falta resolver um problema específico do React: **quando**, exatamente, essa requisição deve ser disparada? Não pode ser toda vez que o componente renderiza (senão vira um ciclo sem fim). A resposta é o hook `useEffect`.

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Rodou!");
}, []);
```

Analogia

O corpo do componente é como a receita de um prato — roda toda vez que alguém pede o prato (a cada renderização). O `useEffect` é uma tarefa que só acontece **uma vez**, ao abrir o restaurante (quando o componente aparece pela primeira vez).

O array `[]` (a "lista de dependências") faz o efeito rodar **uma única vez**.

Atenção

Esquecer o array `[]` faz o efeito rodar a **cada** renderização — pode causar loops.

10. Buscando Dados de uma API Real

O padrão dados / carregando / erro.

src/pages/Usuarios.jsx (arquivo novo)

```
import { useState, useEffect } from "react";

export default function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const [carregando, setCarregando] = useState(true);
  const [erro, setErro] = useState(null);

  useEffect(() => {
    async function buscarUsuarios() {
      try {
        const resposta = await fetch("https://jsonplaceholder.typicode.com/users");
        const dados = await resposta.json();
        setUsuarios(dados);
      } catch (e) {
        setErro("Não foi possível carregar os usuários.");
      } finally {
        setCarregando(false);
      }
    }

    buscarUsuarios();
  }, []);

  if (carregando) {
    return <p>Carregando...</p>;
  }

  if (erro) {
    return <p>{erro}</p>;
  }

  return (
    <div>
      <h1>Usuários</h1>
      <ul>
        {usuarios.map((usuario) => (
          <li key={usuario.id}>{usuario.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

Explicando bloco a bloco

- `useState([])`, `useState(true)`, `useState(null)` — os três estados que toda busca de API costuma precisar: os dados, se ainda está carregando, e se algo deu errado.
- `async function buscarUsuarios() {...}` — por que uma função separada dentro do `useEffect`? Porque o callback do `useEffect` não pode ser `async` diretamente — regra do React. A solução é sempre declarar uma função `async` por dentro e chamá-la em seguida.
- `try / catch / finally` — se a busca der certo, `setUsuarios(dados)` roda; se der errado, cai no `catch`; e o `finally` **sempre** roda, garantindo que `carregando` volte a `false` de qualquer jeito.
- As checagens `if (carregando)` / `if (erro)` vêm antes do retorno final, que só desenha a lista quando os dados já chegaram de verdade.

Ligando ao `App.jsx`

`src/App.jsx` (editar — acrescentar import, link e rota)

```
import Usuarios from "../pages/Usuarios";
...
<Link to="/usuarios">Usuários</Link>
...
<Route path="/usuarios" element={<Usuarios />} />
```

Padrão para memorizar

`dados` / `carregando` / `erro` se repete quase sempre que se busca algo de uma API — vale memorizar essa estrutura.

11. API + Rotas — Buscando por Parâmetro

Juntando useParams com useEffect.

src/pages/UsuarioDetalhe.jsx (arquivo novo)

```
import { useParams } from "react-router-dom";
import { useState, useEffect } from "react";

export default function UsuarioDetalhe() {
  const { id } = useParams();
  const [usuario, setUsuario] = useState(null);
  const [carregando, setCarregando] = useState(true);

  useEffect(() => {
    async function buscarUsuario() {
      const resposta = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
      const dados = await resposta.json();
      setUsuario(dados);
      setCarregando(false);
    }
    buscarUsuario();
  }, [id]);

  if (carregando) {
    return <p>Carregando...</p>;
  }

  return (
    <div>
      <h1>{usuario.name}</h1>
      <p>{usuario.email}</p>
    </div>
  );
}
```

src/pages/Usuarios.jsx (editar – trocar por <Link>)

```
import { Link } from "react-router-dom";
...
<li key={usuario.id}>
  <Link to={`/usuarios/${usuario.id}`}>{usuario.name}</Link>
</li>
```

src/App.jsx (editar – mais um import e uma rota)

```
import UsuarioDetalhe from "../pages/UsuarioDetalhe";
...
<Route path="/usuarios/:id" element={<UsuarioDetalhe />} />
```

Novidade

O array de dependências agora é `[id]`, não `[]` — isso diz ao React: "rode este efeito de novo sempre que `id` mudar", essencial para buscar de novo quando o usuário navega de `/usuarios/1` para `/usuarios/2`.

Resumo geral desta aula

Conceito / Ferramenta	Para que serve
API	"Porta de entrada" para pedir dados a um servidor.
Requisição / Resposta	O pedido que fazemos e os dados que voltam.
JSON	Formato de texto em que os dados costumam vir, parecido com objetos/arrays JS.
Assíncrono	Código que não trava a aplicação enquanto espera algo demorado (como uma resposta de rede).
<code>async / await</code>	Sintaxe para esperar uma resposta assíncrona, escrevendo o código de cima para baixo.
<code>useEffect(() => {...}, [])</code>	Roda código uma vez, após o componente aparecer na tela.
dados / carregando / erro	Os três estados que toda busca de API costuma precisar.

Desenvolvimento Web — Guia de Estudos • Formulários e Fundamentos de API • Projeto de referência: Routes.